

Analytical Problem Solving

Warehouse Robot Search Problem (BFS vs. DFS)

A. PEAS Description

PEAS Element	Description for Warehouse Delivery Robot
Performance Measure	Minimizing total path cost (number of moves/steps taken), successfully reaching cell (5,5) with the package, avoiding obstacles/shelving units, and avoiding infinite loops or off-grid moves.
Environment	A 5×5 grid warehouse matrix with obstacle cells at (2,2), (2,3), (3,3), (4,2), and (4,4). The grid boundary is [1,5]×[1,5].
Actuators	Wheel motors or drive mechanisms enabling moves in 4 cardinal directions: Up, Down, Left, Right.
Sensors	Position sensor/wheel encoders (e.g., GPS, indoor positioning, or odometry) to determine current cell (x,y), and proximity sensors/LiDAR/obstacle maps to detect boundary edges and blocked shelves.

B. Agent Classification

The best suited agent type for this task is a Goal-Based Agent (or a Utility-Based Agent if optimizing path cost dynamically under variable traffic).

Justification: A simple reflex agent operates purely on condition-action rules based on immediate percepts (e.g., "if wall ahead, turn right"). In a maze with obstacles, a simple reflex agent easily gets trapped in infinite loops or dead ends because it lacks foresight. A goal-based agent combines current state perception with explicit goal information—here, reaching cell (5,5)—and plans a sequence of actions (using search algorithms) to safely navigate around blocked shelves to reach that target destination.

C. Formal Search Problem Formulation

State Space: Set of all valid grid coordinates (x,y) where $x,y \in \{1,2,3,4,5\}$ and $(x,y) \in$

Obstacles.

Obstacles = $\{(2,2), (2,3), (3,3), (4,2), (4,4)\}$

Initial State: $(1,1)$

Goal State: $(5,5)$

● Actions: $\text{Actions}(s) \subseteq \{\text{Up}, \text{Down}, \text{Left}, \text{Right}\}$:

1. Up: Moves from (x,y) to $(x-1,y)$
2. Down: Moves from (x,y) to $(x+1,y)$
3. Left: Moves from (x,y) to $(x,y-1)$
4. Right: Moves from (x,y) to $(x,y+1)$

(An action is valid only if the resulting coordinate stays within the 5×5 grid bounds and is not in Obstacles.)

Transition Model: $R(s,a)=s$

where 's' is the resulting coordinate after executing action a from state s.

Path Cost Function: $C(s,a,s)=1$ for each valid move. The total path cost $g(n)$ is the sum of step costs along the path (i.e., the total number of moves).

D. Breadth-First Search (BFS) Execution

Order of action expansion: Up, Down, Left, Right (standard convention; invalid/blocked/visited nodes are ignored).

Let the queue (Frontier) store path nodes as $[\text{Current_State}, \text{Path_So_Far}]$.

Visited set tracks visited states to avoid redundant expansion.

Step-by-Step Expansion:

Step 0:

Pop: $(1,1)$

Frontier: $[(1,2)], [(2,1)]$

Visited: $\{(1,1), (1,2), (2,1)\}$

Step 1:

Pop: (1,2)
Expand (1,2):
Down: (2,2) — Blocked
Left: (1,1) — Visited
Right: (1,3) — Valid
Frontier: [(2,1)], [(1,3)]

Visited: add (1,3)

Step 2:

Pop: (2,1)

Expand (2,1):

Up: (1,1) — Visited

Down: (3,1) — Valid

Right: (2,2) — Blocked

Frontier: [(1,3)], [(3,1)]

Visited: add (3,1)

Step 3:

Pop: (1,3)

Expand (1,3):

Down: (2,3) — Blocked

Right: (1,4) — Valid

Frontier: [(3,1)], [(1,4)]

Visited: add (1,4)

Step 4:

Pop: (3,1)

Expand (3,1):

Down: (4,1) — Valid

Right: (3,2) — Valid

Frontier: [(1,4)], [(4,1)], [(3,2)]

Visited: add (4,1),(3,2)

Step 5:

Pop: (1,4)

Expand (1,4):

Right: (1,5) — Valid

Frontier: [(4,1)], [(3,2)], [(1,5)]

Visited: add (1,5)

Step 6:

Pop: (4,1)

Expand (4,1):

Down: (5,1) — Valid

Right: (4,2) — Blocked

Frontier: [(3,2)], [(1,5)], [(5,1)]

Visited: add (5,1)

Step 7:

Pop: (3,2)

Expand (3,2):

Up: (2,2) — Blocked

Down: (4,2) — Blocked

Right: (3,3) — Blocked

(No new valid neighbors)

Frontier: [(1,5)], [(5,1)]

Step 8:

Pop: (1,5)

Expand (1,5):

Down: (2,5) — Valid

Frontier: [(5,1)], [(2,5)]

Visited: add (2,5)

Step 9:

Pop: (5,1)

Expand (5,1):

Right: (5,2) — Valid

Frontier: [(2,5)], [(5,2)]

Visited: add (5,2)

Step 10:

Pop: (2,5) → Adds (3,5) to frontier

Step 11: Pop (5,2) → Adds (5,3) to frontier

Step 12: Pop (3,5) → Adds (4,5) to frontier

Step 13: Pop (5,3) → Adds (5,4) to frontier

Step 14: Pop (4,5) → Adds (5,5) — Goal reached! (also accessible via (5,4)→(5,5))

BFS Path: (1,1) -> (1,2) -> (1,3) -> (1,4) -> (1,5) -> (2,5) -> (3,5) -> (4,5) -> (5,5)

(Alternative equally optimal path via bottom edge: (1,1) -> (2,1) -> (3,1) -> (4,1) -> (5,1) -> (5,2) -> (5,3) -> (5,4) -> (5,5))

Total Cost: 8 moves

E. Depth-First Search (DFS) Execution

Order of action expansion: Up, Down, Left, Right (pushed onto Stack; top of stack popped next).

Step-by-Step Expansion:

Start: Push (1,1) onto Stack.

Pop (1,1): Neighbors: (1,2) [Right], (2,1) [Down]. Stack pushes Down then Right ⇒ Top is (1,2) or (2,1) depending on insertion order. Using standard move priority order Up, Down, Left, Right:

From (1,1), Down gives (2,1), Right gives (1,2).

- Traversal moves down depth-first along $(2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (5,1) \rightarrow (5,2) \rightarrow (5,3) \rightarrow (5,4) \rightarrow (5,5)$.
- Traversal Sequence: $(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (5,1) \rightarrow (5,2) \rightarrow (5,3) \rightarrow (5,4) \rightarrow (5,5)$
- DFS Path: $(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (5,1) \rightarrow (5,2) \rightarrow (5,3) \rightarrow (5,4) \rightarrow (5,5)$
- Total Cost: 8 moves.

F. Comparison of BFS vs DFS for this Scenario

Metric	BFS	DFS
Completeness	Complete (always finds a solution on finite grids).	Complete (in finite state space with cycle detection).
Optimality	Optimal (guarantees shortest path since step costs are uniform = 1).	Not Optimal in general (depends on branching order; can return deep non-optimal paths).
Time Complexity	$O(b^d)$, where b is branching factor (≤ 4) and d is goal depth (8).	$O(b^m)$, where m is maximum grid depth.
Space Complexity	$O(b \cdot d)$ — holds all frontier nodes per level in memory.	$O(b \cdot m)$ — requires linear memory proportional to max depth.

- Algorithm Suitability: BFS is strictly more suitable for this warehouse delivery problem. Because step costs are uniform (each move costs 1 unit of battery/time), BFS guarantees finding the shortest, most fuel/time-efficient path to cell (5,5). DFS does not guarantee an optimal route and could lead the robot along unnecessary, convoluted detours in larger grid layouts.

Code Implementation

```
AI CO-1.py - C:/Users/ravee/AI CO-1.py (3.13.4)
File Edit Format Run Options Window Help

from collections import deque

# Grid definition
GRID_SIZE = 5
START = (1, 1)
GOAL = (5, 5)
OBSTACLES = {(2, 2), (2, 3), (3, 3), (4, 2), (4, 4)}

# Movements: Up, Down, Left, Right (represented as (row_change, col_change))
MOVES = [(-1, 0), (1, 0), (0, -1), (0, 1)]

def is_valid(r, c):
    return 1 <= r <= GRID_SIZE and 1 <= c <= GRID_SIZE and (r, c) not in OBSTACLES

def bfs(start, goal):
    queue = deque([start])
    visited = {start}

    while queue:
        path = queue.popleft()
        curr = path[-1]

        if curr == goal:
            return path

        r, c = curr
        for dr, dc in MOVES:
            nr, nc = r + dr, c + dc
            if is_valid(nr, nc) and (nr, nc) not in visited:
                visited.add((nr, nc))
                queue.append(path + [(nr, nc)])

    return None

def dfs(start, goal):
    stack = [start]
    visited = {start}

    while stack:
        path = stack.pop()
        curr = path[-1]

        if curr == goal:
            return path

        r, c = curr
        for dr, dc in MOVES:
            nr, nc = r + dr, c + dc
            if is_valid(nr, nc) and (nr, nc) not in visited:
                visited.add((nr, nc))
                stack.append(path + [(nr, nc)])

    return None

# Execution
bfs_path = bfs(START, GOAL)
dfs_path = dfs(START, GOAL)

print(f"BFS Path: {bfs_path} | Cost: {len(bfs_path) - 1}")
print(f"DFS Path: {dfs_path} | Cost: {len(dfs_path) - 1}")
```

OUTPUT:

```
IDLE Shell 3.13.4
File Edit Shell Debug Options Window Help
Python 3.13.4 (tags/v3.13.4:8a526ec, Jun 3 2025, 17:46:04) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ravee/AI CO-1.py =====
BFS Path: [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)] | Cost: 8
DFS Path: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 5), (3, 5), (4, 5), (5, 5)] | Cost: 8
>>>
```

CONCLUSION

- **Goal-Based Agent:** The robot navigates a 5×5 grid from (1,1) to (5,5) while avoiding 5 shelf obstacles using cardinal movements with uniform cost 1.
- **Optimal Path & Cost:** Both BFS and DFS locate a 9-node path—such as (1,1) -> (2,1) -> (3,1) -> (4,1) -> (5,1) -> (5,2) -> (5,3) -> (5,4) -> (5,5)—yielding an optimal cost of 8 moves.
- **Preferred Algorithm:** BFS is ideal for this warehouse environment because it guarantees finding the shortest, most energy-efficient route under uniform step costs.